

QUESTION-14

Constructing a Tree Transversed

main.c	Output
<pre>1 #include <stdio.h> 2 #include <stdlib.h> 3 struct Node 4 { 5 int data; 6 struct Node *left, *right; 7 }; 8 struct Node* createNode(int data) 9 { 10 struct Node *newNode = (struct Node *)malloc(sizeof(struct Node)); 11 newNode->data = data; 12 newNode->left = newNode->right = NULL; 13 return newNode; 14 } 15 void inorder(struct Node *root) 16 { 17 if(root != NULL) 18 { 19 inorder(root->left); 20 printf("%d ", root->data); 21 inorder(root->right); 22 } 23 }</pre>	<pre>Inorder Traversal: 4 2 5 1 3 Preorder Traversal: 1 2 4 5 3 Postorder Traversal: 4 5 2 3 1 === Code Execution Successful ===</pre>

main.c	Output
<pre>24 void preorder(struct Node *root) 25 { 26 if(root != NULL) 27 { 28 printf("%d ", root->data); 29 preorder(root->left); 30 preorder(root->right); 31 } 32 } 33 void postorder(struct Node *root) 34 { 35 if(root != NULL) 36 { 37 postorder(root->left); 38 postorder(root->right); 39 printf("%d ", root->data); 40 } 41 } 42 int main() 43 { 44 struct Node *root = createNode(1); 45 root->left = createNode(2); 46 root->right = createNode(3); 47 root->left->left = createNode(4);</pre>	<pre>Inorder Traversal: 4 2 5 1 3 Preorder Traversal: 1 2 4 5 3 Postorder Traversal: 4 5 2 3 1 === Code Execution Successful ===</pre>

main.c	Output
<pre>33 void postorder(struct Node *root) 34 { 35 if(root != NULL) 36 { 37 postorder(root->left); 38 postorder(root->right); 39 printf("%d ", root->data); 40 } 41 } 42 int main() 43 { 44 struct Node *root = createNode(1); 45 root->left = createNode(2); 46 root->right = createNode(3); 47 root->left->left = createNode(4); 48 root->left->right = createNode(5); 49 printf("Inorder Traversal: "); 50 inorder(root); 51 printf("\nPreorder Traversal: "); 52 preorder(root); 53 printf("\nPostorder Traversal: "); 54 postorder(root); 55 return 0; 56 }</pre>	<pre>Inorder Traversal: 4 2 5 1 3 Preorder Traversal: 1 2 4 5 3 Postorder Traversal: 4 5 2 3 1 === Code Execution Successful ===</pre>